

Day 13. Transformer Architecture – Part 1

Section 1: What is Transformer Architecture?

1.1 The Problem Before Transformers

Before 2017, all sequence-to-sequence tasks (translation, summarization, question answering) were dominated by RNNs, LSTMs, and GRUs. These architectures had a fundamental flaw — they processed tokens one by one, in order. This sequential nature created two critical bottlenecks:

Problem	Impact
Sequential processing	Cannot parallelize — training on GPUs was slow
Vanishing gradient	Long-range dependencies were lost (token 1 forgets token 100)
Fixed context window	RNNs struggled with very long sequences
Information bottleneck	Encoder compressed everything into one vector for the decoder

⇒ Research papers → 2017

key insight was radical: you don't need any recurrence or convolutions at all. Pure attention mechanisms are enough to capture all relationships in a sequence — and they can run in parallel.

- **Authors:** Vaswani et al. — Google Brain & Google Research
- **Year:** 2017 — NeurIPS conference
- **Key claim:** Self-attention can replace RNNs entirely
- **Result:** BERT, GPT, T5, LLaMA, Gemini — all built on this

⇒ "Attention is All you need"

Core Idea

Instead of processing left-to-right, Transformers look at ALL tokens simultaneously and compute how much each token should 'attend to' every other token. This is self-attention — the engine of the Transformer.

1.3 Why Transformer Won

Feature	RNN/LSTM	Transformer
Processing	Sequential (one by one)	Parallel (all at once)
Long-range deps	Poor (gradient vanishes)	Excellent (direct attention)
Training speed	Slow on GPUs	Very fast on GPUs/TPUs
Scalability	Limited	Scales to billions of params
Context length	~100-200 tokens practical	4K to 1M+ tokens

1.4 High-Level Transformer Architecture

The original Transformer is an Encoder-Decoder architecture designed for sequence-to-sequence tasks like translation. Today, many variants exist:

- **Encoder-only:** BERT — good for understanding tasks (classification, NER)
- **Decoder-only:** GPT, LLaMA, Gemini — good for generation tasks
- **Encoder-Decoder:** T5, BART — good for translation, summarization

In Day 13, we cover the full journey into the Encoder: Input → Tokenization → Embedding → Positional Encoding → Encoder Block.

Section 2: Tokenization — Converting Text to Numbers

2.1 Why Tokenization?

Neural networks cannot process raw text — they only understand numbers. Tokenization is the process of converting text into a sequence of integer IDs that the model can work with.

Flow

Raw Text → Tokens → Token IDs → Embeddings → Model

2.2 What is a Token?

A token is not necessarily a word. It can be a whole word, a sub-word, a single character, or even punctuation. The granularity depends on the tokenizer algorithm used.

● "ChatGPT is amazing!"

- Word-level: ['ChatGPT', 'is', 'amazing', '!'] → 4 tokens
- Sub-word (BPE): ['Chat', 'G', 'PT', 'is', 'amazing', '!'] → 6 tokens
- Character-level: ['C', 'h', 'a', 't', 'G', 'P', 'T', ' ', 'i', 's', '!', ...] → 18 tokens

2.3 Types of Tokenization Algorithms

A. Word-Level Tokenization

The simplest approach — split text on spaces and punctuation. Each unique word is a token.

- **Vocabulary:** All unique words in the corpus
- **Problem 1:** Out-of-vocabulary (OOV) words — 'COVID-19' not in vocab → [UNK]
- **Problem 2:** Huge vocabulary size — hundreds of thousands of words
- **Problem 3:** Cannot handle morphological variants — 'run', 'running', 'ran' are 3 separate tokens
- **Used by:** Early NLP models, Word2Vec (not modern LLMs)

B. Character-Level Tokenization

- **Advantage:** No OOV problem — can handle any word
- **Disadvantage:** Very long sequences — 'Transformer' = 11 tokens instead of 1
- **Problem:** Model must learn that 't','r','a','n','s',... together mean 'transformer' — hard to learn meaning
- **Used by:** Some early character-level LMs, rarely used in modern LLMs

C. Sub-Word Tokenization (Modern Standard)

The sweet spot between word-level and character-level. Splits rare or unknown words into smaller known pieces, but keeps common words as single tokens.

Key Insight
 'unbelievably' → ['un', 'believ', 'ably'] — the model can understand prefix 'un' means negation, which helps generalize to new words it has never seen.

D. Byte Pair Encoding (BPE) — Used in GPT, LLaMA, Gemini

BPE was originally a data compression algorithm adapted for NLP by Sennrich et al. (2016). It builds vocabulary bottom-up by merging the most frequent pairs of characters or sub-words.

BPE Algorithm Step-by-Step:

1. Start with character-level vocabulary: ['l','o','w','e','r','n','s','t...']
2. Count all adjacent pair frequencies in the corpus
3. Merge the most frequent pair into a new token ('l'+o' → 'lo')
4. Repeat until desired vocabulary size is reached (e.g., 50,000 tokens)

Model	Vocabulary Size (BPE)
GPT-2	50,257 tokens
GPT-3 / GPT-4	~100,000 tokens (cl100k_base)
LLaMA 2	32,000 tokens
LLaMA 3	128,256 tokens
Gemma	256,000 tokens

E. WordPiece Tokenization — Used in BERT

WordPiece is similar to BPE but uses a different merge criterion. Instead of frequency, it maximizes the likelihood of the training data given the vocabulary.

- Words are split using '##' prefix for sub-words that are not the start of a word
- Example: 'playing' → ['play', '##ing']
- Example: 'unaffordable' → ['un', '##afford', '##able']
- Vocabulary:** BERT uses ~30,000 WordPiece tokens
- Special tokens:** [CLS], [SEP], [MASK], [PAD], [UNK]

🔍 BPE vs WordPiece

BPE merges based on frequency (most common pairs). WordPiece merges based on likelihood gain. Both produce similar results in practice. The key difference: WordPiece tends to keep linguistically meaningful subwords.

F. SentencePiece — Used in T5, LLaMA

SentencePiece treats the input as a raw byte stream and does not require pre-tokenization (no whitespace splitting first). It works directly on raw text including whitespace as part of tokens.

- Language-agnostic — works for Japanese, Chinese, Hindi without spaces
- Treats space as a special character '___' (underscore)
- Example: '___Hello ___world' instead of 'Hello world'
- Used by:** T5, mT5, LLaMA, Mistral, PaLM

G. Tiktoken — OpenAI's BPE Implementation

Tiktoken is OpenAI's fast BPE tokenizer used in GPT-3.5, GPT-4. It uses byte-level BPE, meaning every byte (0-255) is in the base vocabulary — guaranteeing no OOV ever.

- cl100k_base:** Used in GPT-3.5 and GPT-4 — 100,277 tokens
- p50k_base:** Used in GPT-2 and Codex — 50,281 tokens
- Tiktoken is written in Rust — extremely fast tokenization speed

🎮 2.4 Special Tokens

Modern tokenizers add special tokens to control model behavior and structure

Token	Model	Purpose
[CLS]	BERT	Classification token — placed at start, output used for classification
[SEP]	BERT	Separator — marks boundary between two sentences
[MASK]	BERT	Masked language modeling — token to be predicted
[PAD]	BERT/GPT	Padding — fills shorter sequences in a batch
<BOS> / <s>	GPT / LLaMA	Beginning of sequence marker
<EOS> / </s>	GPT / LLaMA	End of sequence — model stops generating
<UNK>	Most models	Unknown token for OOV words (rare in sub-word)

Section 3: Embeddings — From Token IDs to Meaning Vectors

3.1 Why Do We Need Embeddings?

After tokenization, we have a list of integer IDs like [101, 7592, 1010, 1045, 2293, 4103, ...]. These numbers are arbitrary labels — token 7592 is not 'greater than' token 101. We need to convert these IDs into dense vectors that capture semantic meaning.

💡 Analogy

Think of each token as a person. An embedding is their 'personality profile' — a list of 768 (or 1024 or 4096) numbers that describe who they are in meaning-space. Words with similar meanings end up with similar profiles.

3.2 What is an Embedding?

An embedding is a dense, low-dimensional vector representation of a discrete token. Instead of a one-hot vector (vocabulary_size = 50,000 dimensions, mostly zeros), we use a compact vector of 256 to 4096 dimensions.

Representation	Properties
One-hot encoding	50,000 dims, sparse, no semantic info, 'cat'≠'dog' but also 'cat'≠'kitten'
Word embeddings	256-4096 dims, dense, semantic relationships

captured

The embedding matrix E has shape [vocabulary_size × embedding_dimension]. Looking up a token is just indexing into this matrix: $E[\text{token_id}]$ gives the embedding vector.

3.3 Types of Embeddings

A. Static / Non-Contextual Embeddings (Pre-Transformer Era)

Each word has ONE fixed embedding regardless of context. The word 'bank' has the same vector whether it means river bank or financial bank.

Word2Vec (2013) — Google

Two architectures for learning word embeddings from large text corpora:

Word2Vec (2013) — Google

Two architectures for learning word embeddings from large text corpora:

- **CBOW (Continuous Bag of Words):** Predict center word from surrounding context words
- **Skip-gram:** Predict surrounding context words from center word
- **Vector dimension:** Typically 100-300 dimensions
- **Famous property:** King - Man + Woman = Queen (linear vector arithmetic)
- **Limitation:** One vector per word — no context awareness

GloVe — Global Vectors (2014) — Stanford

GloVe learns embeddings by factorizing a word co-occurrence matrix built from a large corpus.

- Combines global matrix factorization + local context window
- Pre-trained on Common Crawl (840B tokens), Wikipedia
- Available in 50, 100, 200, 300 dimension versions
- Still widely used for lightweight NLP tasks

FastText (2016) — Facebook AI

Extension of Word2Vec that uses character n-grams within words.

- 'playing' = 'play' + 'layi' + 'ayin' + 'ying' + ... (all 3-6 char n-grams)
- Can generate embeddings for OOV words from their characters
- Better for morphologically rich languages (German, Hindi, Turkish)

B. Contextual Embeddings (Transformer Era)

The same word gets DIFFERENT embeddings depending on its context. This is the breakthrough of Transformer-based models.

Example

'bank' in 'I went to the river bank' → embedding shifted toward water/geography. 'bank' in 'I deposited money at the bank' → embedding shifted toward finance/institution.

ELMo — Embeddings from Language Models (2018) — AllenAI

First major contextual embedding model. Used bidirectional LSTMs to create context-aware representations.

- Runs text through forward and backward LSTM
- Concatenates hidden states from multiple LSTM layers
- Pre-trained on 1 billion word benchmark
- Still sequential — not parallel like Transformers

BERT Embeddings (2018) — Google

BERT (Bidirectional Encoder Representations from Transformers) generates contextual embeddings using the full Transformer encoder. The [CLS] token embedding is used for classification tasks.

- bert-base: 768-dimensional embeddings, 12 encoder layers
- bert-large: 1024-dimensional embeddings, 24 encoder layers
- Same word → different 768-dim vector depending on full sentence context

GPT Embeddings — OpenAI

Decoder-only transformer. The final hidden state of each token is its contextual embedding.

GPT embeddings are primarily used for text generation, not standalone embedding lookup.

Modern Dedicated Embedding Models

Model	Embedding Dim & Use Case
text-embedding-3-small (OpenAI)	1536 dims — fast, cheap retrieval
text-embedding-3-large (OpenAI)	3072 dims — high quality RAG
sentence-transformers (SBERT)	384-768 dims — semantic similarity
Cohere Embed v3	1024 dims — multilingual RAG
Gemini text-embedding-004	768 dims — Google ecosystem

 3.4 Token Embedding in the Transformer

Inside the Transformer, the embedding layer is simply a learned lookup table. During training, this matrix is updated via backpropagation — the model learns which vectors best represent each token for the task at hand.

- **Input:** Token ID (integer, e.g., 7592)
- **Process:** Index into embedding matrix E of shape [V × d_model]
- **Output:** Dense vector of shape [d_model] (e.g., 512 or 768)
- **d_model in original paper:** 512 dimensions
- **Scaling:** Embeddings are multiplied by $\sqrt{d_model}$ to stabilize training



Section 4: Positional Encoding — Injecting Order into Embeddings

4.1 Why Do We Need Positional Encoding?

Self-attention treats all tokens as a set — it has no concept of order. 'The cat sat on the mat' and 'Mat the on sat cat the' would produce IDENTICAL attention outputs because attention is permutation-equivariant.

Problem

Without positional encoding, 'Dog bites man' and 'Man bites dog' are the same to the Transformer. Order is completely lost. Position must be explicitly injected into the input.

4.2 Types of Positional Encoding

A. Absolute Sinusoidal Positional Encoding (Original Transformer)

The original 'Attention Is All You Need' paper uses fixed (non-learned) sinusoidal functions to create positional encodings. For position pos and dimension i :

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

pos = position of the token in the sequence (0, 1, 2, ...)
i = dimension index (0 to d_model/2)
Source: <https://arxiv.org/pdf/1609.08144v2.pdf>

Why sinusoidal? Three key reasons:

- Different frequencies allow the model to attend to different ranges (short, medium, long distances)
- $PE(pos+k)$ can be expressed as a linear function of $PE(pos)$ — relative positions are implicitly learnable
- Works for sequences longer than those seen during training (extrapolates)



Intuition

Think of it like a clock. Position 0 = 12:00:00. Position 1 = 12:00:01. Each position has a unique pattern of sine waves at different frequencies — like a fingerprint. The model learns to read these fingerprints to understand order.

B. Learned Absolute Positional Embeddings

Instead of computing positions with fixed formulas, make positional encodings trainable parameters. Each position index gets its own embedding vector, learned from data.

- **BERT uses:** Learned positional embeddings (max 512 positions)
- **GPT-1/GPT-2 use:** Learned positional embeddings
- **Advantage:** Model learns the most useful positional patterns for the task

- **Disadvantage:** Cannot generalize beyond max trained sequence length

C. Relative Positional Encoding

Instead of encoding absolute positions, encode the relative distance between tokens. Token at position 5 doesn't know it's at position 5 — it knows it's 3 positions after token at position 2.

- **Shaw et al. (2018):** Self-Attention with Relative Position Representations
- Modifies attention score computation: adds learned relative position bias to $Q \cdot K^T$
- **T5 uses:** Relative position biases — bucket-based relative positions
- **Advantage:** Better generalization to different sequence lengths

D. ALiBi — Attention with Linear Biases (2022)

ALiBi adds a fixed linear penalty to attention scores based on the distance between tokens. No positional embeddings are added to word embeddings at all.

- Attention score modified: $\text{softmax}(Q \cdot K^T / \sqrt{d} + m \cdot [-\infty, \dots, -2, -1, 0])$
- Each head has a different slope m — multi-head learns different distance sensitivities
- **Used by:** MPT, BLOOM
- **Key advantage:** Extrapolates to much longer sequences than trained on

E. RoPE — Rotary Position Embedding (2021)

RoPE encodes position by rotating the query and key vectors in complex space by an angle proportional to their position. This makes the dot product $Q \cdot K$ naturally encode relative position.

- Applied to Q and K directly during attention computation
- Mathematically: multiply each pair of dimensions by a rotation matrix based on position
- **Used by:** LLaMA 1/2/3, Mistral, Qwen, Falcon, PaLM 2, Gemma
- **Key advantage:** Elegant, computationally efficient, great long-context performance
- **Extended RoPE:** YaRN, LongRoPE — extend context to 128K+ tokens

Section 5: Complete Encoder Architecture

5.1 Overview of the Encoder

The Transformer encoder takes a sequence of token embeddings (already summed with positional encodings) and outputs a sequence of contextualized representations — the same number of vectors, but each one now understands its context within the full sequence.

- **Input:** Sequence of vectors $[x_1, x_2, \dots, x_n]$ each of shape $[d_model]$
- **Output:** Sequence of vectors $[z_1, z_2, \dots, z_n]$ each of shape $[d_model]$
- **The encoder in original paper:** Stack of $N=6$ identical layers
- **BERT-base:** $N=12$ encoder layers
- **BERT-large:** $N=24$ encoder layers

5.2 Inside One Encoder Layer

Each encoder layer has exactly two sub-layers:

5. **Multi-Head Self-Attention (MHSA)**
6. **Position-wise Feed-Forward Network (FFN)**

Both sub-layers use a Residual Connection + Layer Normalization. The formula for each sub-layer output:

Output = LayerNorm(x + Sublayer(x))

x = Input to the sub-layer
Sublayer(x) = MHSA(x) or FFN(x)
LayerNorm = normalized version of x

5.3 Sub-Layer 1: Multi-Head Self-Attention (MHSA)

(Covered in depth in Day 12 — quick recap here for completeness)

- Output = weighted sum of Values

Multi-Head Attention

Instead of one attention computation, run h parallel attention heads, each with different learned projections:

- **h = number of heads:** 8 in base model, 12 in BERT-base, 16 in BERT-large
- **d_k = d_v = d_model / h:** 64 dimensions per head when d_model=512, h=8
- Concatenate outputs of all heads → project back to d_model
- Each head learns to attend to different types of relationships (syntax, coreference, semantics...)

💡 Why Multi-Head?

One head might learn to match subjects to verbs. Another head learns to match pronouns to their referents. Another tracks semantic similarity. Multi-head = multiple parallel 'relationship detectors' running simultaneously.

5.4 Sub-Layer 2: Feed-Forward Network (FFN)

After attention aggregates information across tokens, the FFN processes each token independently — adding non-linearity and capacity to the model.

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x} \cdot \mathbf{W}_1 + b_1) \cdot \mathbf{W}_2 + b_2$$

- **d_ff**: Inner dimension = $4 \times d_{\text{model}}$ (2048 for base, 3072 for BERT-base, 4096 for BERT-large)
- Applied position-wise: same FFN applied independently to each token position
- No token-to-token interaction here — that was handled by attention
- ReLU in original paper; modern models use GELU, SwiGLU, GeGLU instead

💡 What Does FFN Actually Learn?

Research by Geva et al. (2021) shows that FFN layers act as key-value memory stores — they memorize factual knowledge. 'Paris is the capital of France' type facts are stored in FFN weights. Attention handles syntax/relationships; FFN stores facts.

▶ 5.5 Add & Norm — Residual Connections + Layer Normalization

Residual Connection (Skip Connection)

The input to each sub-layer is added back to its output before normalization: $\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$

- Borrowed from ResNet — prevents vanishing gradient in deep networks
- Ensures gradients can flow directly through the network during backpropagation
- Allows the model to 'skip' a layer if it learns nothing useful (identity function)
- Essential for stacking 12, 24, 48+ encoder layers without gradient problems

Layer Normalization

LayerNorm normalizes the activations across the feature dimension (not the batch dimension like BatchNorm). For each token vector x :

- Compute mean μ and variance σ^2 across all d_{model} dimensions of x
- Normalize: $\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \epsilon)}$

- Scale and shift: $\text{output} = \gamma \cdot \hat{x} + \beta$ (γ and β are learned parameters)
- **Why not BatchNorm?** Batch size varies; LayerNorm is independent of batch size — works well for sequences

💡 Pre-LN vs Post-LN

Original paper uses Post-LN: $\text{LayerNorm}(x + \text{Sublayer}(x))$. Most modern models use Pre-LN: $x + \text{Sublayer}(\text{LayerNorm}(x))$. Pre-LN is more stable during training and is used in GPT-2, LLaMA, and most modern LLMs.

5.6 Complete Encoder Layer — Full Data Flow

Here is the complete step-by-step data flow through one encoder layer:

Step	Operation	Shape / Notes
1	Input x	$[\text{seq_len} \times d_{\text{model}}]$ — token embeddings + positional encoding
2	MHSA: Q,K,V projection	$[\text{seq_len} \times d_{\text{model}}] \rightarrow h \text{ heads of } [\text{seq_len} \times d_{\text{k}}]$
3	MHSA: Scaled dot-product attention	Attention(Q,K,V) per head $\rightarrow [\text{seq_len} \times d_{\text{v}}]$
4	MHSA: Concatenate + project	$[\text{seq_len} \times (h \cdot d_{\text{v}})] \rightarrow [\text{seq_len} \times d_{\text{model}}]$
5	Add & Norm (first)	LayerNorm($x + \text{MHSA_output}$) $\rightarrow [\text{seq_len} \times d_{\text{model}}]$
6	FFN: Expand	$[\text{seq_len} \times d_{\text{model}}] \rightarrow [\text{seq_len} \times d_{\text{ff}}]$ (W_1 , ReLU)
7	FFN: Contract	$[\text{seq_len} \times d_{\text{ff}}] \rightarrow [\text{seq_len} \times d_{\text{model}}]$ (W_2)
8	Add & Norm (second)	LayerNorm($x + \text{FFN_output}$) $\rightarrow [\text{seq_len} \times d_{\text{model}}]$
9	Output to next layer	$[\text{seq_len} \times d_{\text{model}}]$ — richer contextual representations

5.7 Stacked Encoder — N Layers

The output of one encoder layer becomes the input of the next. After N layers, each token's representation has been refined N times — incorporating information from the entire sequence at multiple levels of abstraction.

- **Layer 1:** Learns low-level patterns — syntax, local dependencies
- **Middle layers:** Learns phrase-level structure, grammatical roles
- **Top layers:** Learns semantic meaning, coreference, long-range relationships

Model	$d_{\text{model}} \cdot \text{Heads} \cdot \text{Layers} \cdot d_{\text{ff}}$
Original Transformer (base)	$512 \cdot 8 \text{ heads} \cdot 6 \text{ layers} \cdot 2048$
BERT-base	$768 \cdot 12 \text{ heads} \cdot 12 \text{ layers} \cdot 3072$

BERT-large	$1024 \cdot 16 \text{ heads} \cdot 24 \text{ layers} \cdot 4096$
GPT-2 (small)	$768 \cdot 12 \text{ heads} \cdot 12 \text{ layers} \cdot 3072$
GPT-3 (175B)	$12288 \cdot 96 \text{ heads} \cdot 96 \text{ layers} \cdot 49152$
LLaMA 3 (8B)	$4096 \cdot 32 \text{ heads} \cdot 32 \text{ layers} \cdot 14336$